



CS 305 Project One Template

Document Revision History

Version	Date	Author	Comments
1.0	05-24-2026	Nicole Snyder	

Client



Instructions

Submit this completed vulnerability assessment report. Replace the bracketed text with the relevant information. In this report, identify your security vulnerability findings and recommend the next steps to remedy the issues you have found.

- Respond to the five steps outlined below and include your findings.
- Respond using your own words. You may also include images or supporting materials. If you include them, make certain to insert them in the relevant locations in the document.
- Refer to the Project One Guidelines and Rubric for more detailed instructions about each section of the template.



Developer

Nicole Snyder

1. Interpreting Client Needs

Determine your client's needs and potential threats and attacks associated with the company's application and software security requirements. Consider the following questions regarding how companies protect against external threats based on the scenario information:

- What is the value of secure communications to the company?
- Are there any international transactions that the company produces?
- Are there governmental restrictions on secure communications to consider?
- What external threats might be present now and in the immediate future?
- What modernization requirements must be considered, such as the role of open-source libraries and evolving web application technologies?

A. Value of Secure Communications:

Financial applications handle sensitive information (customer data, transactions). Secure communication ensures confidentiality, integrity, and trust, protecting against interception and tampering.

B. International Transactions?

Yes. As a financial services application handling sensitive consulting data and transactions, secure communication channels are fundamentally mandatory. Modern financial entities operate within a global ecosystem where data routing, cloud-hosted infrastructures, and compliance frameworks routinely cross international borders. Securing these communication channels ensures protection against sophisticated global threat vectors and advanced persistent threats (APTs), regardless of the physical residency of the client base.

C. Governmental Restrictions:

Financial data often falls under regulations like GDPR (EU), GLBA (US), or PCI DSS (global for payment). Compliance may require strong encryption and proper audit trails.

D. External Threats:

SQL Injection via unsanitized inputs

Credential theft (brute-force or phishing)

Man-in-the-middle attacks (if encryption isn't enforced)

Dependence on vulnerable open-source libraries

Denial of Service (DoS)

E. Modernization Requirements:

Open-source libraries: You use spring-boot, json-path, and bcprov-jdk15on. Open-source libraries need regular updates because they're common attack targets.

Evolving web tech: New web apps often use RESTful services, microservices, JSON APIs, and must adapt to new security models.

2. Areas of Security



Refer to the vulnerability assessment process flow diagram. Identify which areas of security apply to Artemis Financial's software application. Justify your reasoning for why each area is relevant to the software application.

Based on a comprehensive architectural review of Artemis Financial's web-based application, four core macro-level areas from the Vulnerability Assessment Process Flow Diagram are directly applicable to the immediate software scope:

- **Input Validation:** The application actively ingests user data through HTTP requests using `@RequestParam` bindings (e.g., `/read`, `/greeting`). Strict input validation is paramount to ensure that untrusted user data is sanitized before hitting internal components, mitigating injection risks.
- **APIs:** The software architecture relies heavily on REST endpoints and controllers to handle CRUD functionality and expose business services. Securing these API interactions is a primary defensive vector to prevent unauthorized endpoint traversal and data exposure.
- **Cryptography:** The codebase explicitly imports the Bouncy Castle cryptographic provider (`bcprov-jdk15on`) to facilitate secure communications and data verification. Ensuring that these cryptographic implementations utilize secure, invulnerable algorithms is a critical requirement for financial record integrity.
- **Code Error Handling:** The manual code review revealed that current implementation patterns handle exceptions by printing stack traces directly to console/logs. Implementing secure error handling is necessary to ensure the application fails gracefully without leaking structural system details to potential attackers.

Scope Exclusions: In accordance with the vulnerability assessment guidelines, not all seven security areas are prioritized for this software-layer report. Transport-layer items (Client/Server) are treated as infrastructure responsibilities, while Encapsulation and generic Code Quality represent secondary object-oriented formatting standards rather than active, high-severity exploit boundaries for this remediation cycle.

3. Manual Review

Continue working through the vulnerability assessment process flow diagram. Identify all vulnerabilities in the code base by manually inspecting the code.

A manual inspection of the project's Java source code and configuration files revealed several potential security vulnerabilities and poor practices:

Hard-coded Database Credentials:

In `DocData.java`, the database username and password are hardcoded directly into the code. This exposes sensitive credentials to anyone with source code access and increases the risk of credential leaks. Database credentials should be externalized through environment variables or a secure configuration service.



Lack of Input Validation on REST Endpoints:

In classes such as `CRUDController.java`, user input received through endpoints (e.g., `@RequestParam("business_name")`) is accepted without validation or sanitization. This makes the application susceptible to injection attacks, such as SQL Injection or script injection, and to logic errors from malformed input.

No Authentication or Authorization Controls:

REST endpoints are publicly accessible without any authentication or role-based access control. This means any user can potentially read, write, or modify sensitive data via the API. All endpoints should require proper authentication and be protected according to user roles and actions.

Poor Exception Handling:

Throughout the code, especially in `DocData.java`, caught exceptions are handled by simply printing the stack trace. This practice can leak internal implementation details in logs or, if not properly secured, to users. Exceptions should be logged securely, and only generic, non-sensitive error messages should be returned to the client.

Outdated Cryptography Libraries:

The project's `pom.xml` includes older versions of cryptographic libraries, such as Bouncy Castle (`bcprov-jdk15on`), which contain known vulnerabilities. Dependency management should ensure all libraries remain up-to-date to avoid exposure to published security flaws.

Incomplete Method Implementations and Missing Validation:

In `myDateTime.java`, methods like `retrieveDateTime()` and `setMyDateTime()` are either not implemented or lack validation logic. This allows undefined or incorrect states to be set, which could lead to functional or security issues.

Unrestricted Internal Fields:

In the `customer.java` class, fields like `account_balance` are package-private (have no access modifier) and are not encapsulated by getters or setters. This can allow unwanted or risky modifications of critical values from elsewhere in the code.

No Enforcement of HTTPS:

There is no configuration or code enforcing the use of HTTPS/TLS for client-server communication. This potentially allows attackers to intercept or modify data in transit, especially sensitive financial or authentication information.

Lack of CSRF Protection:

The application's REST endpoints do not implement Cross-Site Request Forgery (CSRF) protection. Without this, attackers could trick authenticated users into performing unwanted actions on the server.

No Security-Relevant Logging or Auditing:

There is no evidence of security or audit logging for sensitive events such as user logins, failed authentication attempts, or critical data changes. Proper logging and audit trails are necessary for detecting and responding to security incidents.



These findings indicate a need for improved secure coding practices, input validation, authentication enforcement, error management, use of up-to-date dependencies, and operational controls in the application.

4. Static Testing

Run a dependency check on Artemis Financial's software application to identify all security vulnerabilities in the code. Record the output from the dependency-check report. Include the following items:

- The names or vulnerability codes of the known vulnerabilities
- A brief description and recommended solutions provided by the dependency-check report
- Any attribution that documents how this vulnerability has been identified or documented previously

CVE-2016-1000338

- **Description:** In Bouncy Castle JCE Provider version 1.55 and earlier, DSA verification does not fully validate ASN.1 encoding. Attackers may inject extra elements into signature sequences, in some cases allowing invalid signatures to validate (CWE-347).
- **Recommended Solution:** Upgrade bouncycastle:legion-of-the-bouncy-castle-java-cryptography-api to 1.56 or later.
- **Attribution:** NVD, Dependency-Check Report.

CVE-2020-15522

- **Description:** In Bouncy Castle EC Java before 1.66, a timing issue allows attackers to observe signature generation and infer private keys via ECDSA signatures.
- **Recommended Solution:** Upgrade bouncycastle:bouncy_castle_crypto_package_for_java to 1.66 or later.
- **Attribution:** NVD, Dependency-Check Report.

CVE-2023-33202

- **Description:** Bouncy Castle for Java before 1.73 can be exploited for a potential denial of service (DoS) if crafted ASN.1 data is sent to the PEMParser, triggering an OutOfMemoryError.
- **Recommended Solution:** Upgrade bouncycastle:bouncy_castle_crypto_package_for_java to 1.73 or later.
- **Attribution:** NVD, Dependency-Check Report.

CVE-2015-7940

- **Description:** Bouncy Castle Java library before 1.51 does not validate elliptic curve points, making it easier for remote attackers to obtain private keys through "invalid curve attacks."
- **Recommended Solution:** Upgrade bouncycastle:bouncy_castle_crypto_package to 1.51 or later.
- **Attribution:** NVD, Dependency-Check Report.

Outdated Third-Party Dependency: Hibernate Validator Framework

- **Description:** The project's build configuration specifies the use of hibernate-validator-6.0.18.Final.jar for bean and model validation. Static dependency scanning flags this older library version as outdated and highly susceptible to known security vulnerabilities due to a lack of modern security patches.
- **Recommended Solution:** Update the dependency declaration within the project's pom.xml file to a fully patched, actively maintained stable release (such as version 6.2.0.Final or higher, or migration to the 8.x branch depending on Jakarta EE compatibility). Ensure full regression and unit testing are executed post-upgrade to verify validation constraints remain intact.
- **Attribution:** Dependency-Check HTML Report Evidence / National Vulnerability Database (NVD).

CVE-2025-35036

- **Description:** Hibernate Validator before versions 6.2.0 and 7.0.0 may allow code injection by interpolating user-supplied input with Expression Language (EL) in constraint violation messages. Attackers could access sensitive information or execute code if user input is unsafely included.
- **Recommended Solution:** Upgrade to Hibernate Validator 6.2.0 or later. Avoid using user input in constraint violation messages unless properly sanitized.
- **Attribution:** NVD entry on CVE-2025-35036, Dependency-Check report.

CVE-2023-1932

- **Description:** A flaw in the isValid method (hv.SafeHtmlValidator) allows attackers to bypass HTML validation by omitting the end tag (>), resulting in possible HTML injection or XSS.
- **Recommended Solution:** Upgrade to Hibernate Validator 6.2.0 or later where this bug is fixed.
- **Attribution:** NVD entry on CVE-2023-1932.

CVE-2020-10693

- **Description:** A bug in Hibernate Validator 6.1.2.Final's message interpolation processor enables invalid EL expressions to be evaluated, making it possible to bypass input sanitization measures.



- **Recommended Solution:** Upgrade to at least Hibernate Validator 6.0.20 or a more recent version.
- **Attribution:** NVD entry on CVE-2020-10693.

CVE-2025-15104

- **Description:** Nu Html Checker (validator.nu) allows attackers to perform Server-Side Request Forgery (SSRF) by exploiting hostname-based protections (including localhost/127.0.0.1), which can be bypassed using DNS rebinding or specially-crafted domains. This could allow the server to access internal resources.
- **Recommended Solution:** Upgrade validator.nu to a version released after January 11, 2026, with the SSRF fix implemented.
- **Attribution:** NVD entry on CVE-2025-15104

5. Mitigation Plan

Interpret the results from the manual review and static testing report. Then identify the steps to mitigate the identified security vulnerabilities for Artemis Financial's software application.

To address the security issues identified during both static analysis and the manual review, the following mitigation steps are recommended for Artemis Financial's web-based application:

First, all hard-coded secrets such as database usernames and passwords should be removed from the source code. Instead, credentials should be stored in environment variables or secure configuration servers so they are not exposed in source control or application binaries. This will significantly reduce the risk of accidental disclosure of sensitive data.

Second, strong input validation and sanitization should be enforced on all incoming user data, especially for all REST endpoints. Inputs should be checked for correct length, type, format, and content. Leveraging Spring validation annotations or manual checks helps prevent injection attacks, cross-site scripting (XSS), and application errors.

Third, access to all endpoints must be restricted using robust authentication mechanisms, such as Spring Security. Sensitive operations should require users to log in and prove their identity, and authorization controls should ensure only authorized users can access or modify sensitive data.

Fourth, exception handling should be improved throughout the application. Instead of printing stack traces or exposing error details to end users, all exceptions should be caught and logged securely using a logging framework. Clients should only receive generic, non-sensitive error messages.

Fifth, all third-party dependencies, especially cryptography libraries such as Bouncy Castle and Hibernate Validator, must be updated to the latest secure versions recommended by dependency scanning tools. Developers should also regularly monitor for new vulnerabilities and apply patches as they become available.



Sixth, incomplete and unvalidated methods should be fully implemented, including proper handling and validation of all input data. This will help avoid unpredictable behaviors and logic errors that could be exploited by attackers.

Seventh, all sensitive fields in application classes should be made private and accessed only through getters and setters that include appropriate validation and access controls. This reduces the likelihood of accidental or malicious modification of critical application state.

Eighth, HTTPS/TLS should be strictly enforced for all HTTP traffic to the application by configuring the server to redirect all requests to HTTPS and only exposing secure ports. This prevents attackers from eavesdropping or tampering with sensitive financial data in transit.

Ninth, Cross-Site Request Forgery (CSRF) protection should be enabled, especially for endpoints that change server state. Spring Security and related frameworks provide built-in CSRF controls, which should be configured and tested as part of the security review.

Lastly, the application should implement detailed audit and security event logging. All authentication attempts, critical data changes, and access violations should be logged and regularly reviewed. Logging should also trigger alerts to notify administrators of suspicious or malicious activity.

By implementing these mitigations, Artemis Financial's web-based application will significantly reduce its risk exposure and meet industry-standard software security practices.